



SMART CONTRACT AUDIT

Conducted for: Precious Hour



VULSIGHT

Contents

Executive Summary:	3
Project Background:	3
Audit Scope:	3
Audit Summary:	3
Severity Definitions:	4
Technical Checks:	4
Code Quality and Documentation:	5
Audit Findings:	5
Critical vulnerabilities:	5
High vulnerabilities:	5
Mint Reentrancy:	5
Medium vulnerabilities:	6
Weekly Limit Not enforced properly:	6
Low vulnerabilities:	6
Lack of Events:	7
Disclaimer:	7
Conclusion:	7



VULSIGHT

Executive Summary:

The client contacted our Firm to conduct a smart contract audit on their Solidity based smart contract. The smart contract wasn't deployed yet on any chain. The smart contract named "PreciousHour" is a standard implementation of an ERC721 contract with a set of unique features and customizations. It utilizes multiple libraries and extensions from the OpenZeppelin contracts library - a well-audited and widely-accepted set of libraries in the Ethereum community. The contract deploys a set of extensions and libraries, including ERC721Enumerable, ReentrancyGuard, Ownable, SafeMath, and Strings, which collectively provide functionality for token enumeration, reentrancy protection, owner-based access control, safe arithmetic operations, and string conversion respectively. The audit was performed using both manual code analysis and automated security tools. The audit is performed on a best effort basis. Vulnerabilities were detected within the smart contract code during the course of this audit. All of the findings of the audit is detailed inside this report.

Project Background:

The smart contract is written in solidity. The smart contract named "PreciousHour" is a standard implementation of an ERC721 contract with a set of unique features and customizations. This contract includes several libraries and custom features to make it more versatile and secure.

Audit Scope:

Deployed Address	N/A
Chain	N/A
Language	Solidity
Audit Completion Date	17/6/23

Audit Summary:

The **PreciousHour** contract inherits from the **OpenZeppelin ERC721Enumerable**, **ReentrancyGuard**, and **Ownable** contracts, providing standard ERC721 functionality with built-in protection against re-entrancy attacks and an access control mechanism.

The primary purpose of this contract is to mint ERC721 tokens which represent unique NFTs, with a limit on the number that can be minted in a week. It has additional functionality to modify the **timezone** offset and extract a current week. However, it is important to note that the concept of a weekly limit in the contract does not appear to reset every week.

In this review, we analyzed the contract for security risks, code quality, and adherence to best practices. Key areas of consideration included access controls, handling of user funds, and potential reentrancy attacks.

Various tools such as Slither were used in the audit. Extensive manual code review was also done to ensure that any vulnerability if present, could be detected in the audit.

We found:

- **0 Critical risk vulnerabilities**
- **1 High risk vulnerability**
- **1 Medium risk vulnerability**
- **1 Low risk vulnerability**

Severity Definitions:

Risk Level	Description
Critical	Any kind of vulnerability that could lead to direct token or monetary loss
High	Any type of vulnerability that could disrupt the proper functioning of smart contract or indirectly lead to token or monetary loss.
Medium	Any type of vulnerability that could cause undesired actions but no serious disruption or monetary loss
Low	Any type of vulnerability that doesn't have any significant impact on the execution of the proper functioning of contract

Technical Checks:

Checks	Result
Solidity version not specified	Passed
Solidity version too old	Passed
Integer overflow/underflow	Passed
Function input parameters check bypass	Passed
Insufficient randomness used	Passed
Fallback function misuse	Passed
Race Condition	Passed
Logical Vulnerabilities	Not Passed
Event Emission	Not Passed
Reentrancy	Not Passed
Function visibility not explicitly declared	Passed
Use of deprecated keywords/functions	Passed
Out of Gas issue	Passed
Front Running	Passed

Insufficient Decentralization	Passed
Function input parameters lack of check	Passed
Proper function Access Control	Passed
Hardcoded Data	Passed

Code Quality and Documentation:

The audit scope included one smart contract which was written in Solidity programming language. The code is well indented and clear in nature. While the Code lacked commenting. It is recommended that proper commenting be added to aid developers.

Audit Findings:

Critical vulnerabilities:

0 critical vulnerabilities were found in the smart contract.

High vulnerabilities:

1 high vulnerability was found in the smart contract.

Mint Reentrancy:

One major concern raised during the audit pertains to the potential risk of a reentrancy attack associated with the **mint()** function in the contract. The root of the vulnerability lies in the function's external call to **_safeMint()**.

Reentrancy vulnerabilities arise when a contract makes an external call to another contract before it has resolved its state. In this case, the **mint()** function calls the **_safeMint()** function prior to updating all state variables. This order of operations opens the door for a potential reentrancy attack.

In this scenario, the sequence of events would be as follows:

1. An invocation to **mint()**, which internally calls **_safeMint()**.
2. **_safeMint()**, incorporating an external call, calls into another potentially malicious contract.
3. This malicious contract can then invoke **mint()** again.
4. As **tokensSold** is not updated until after **_safeMint()** completes, the malicious contract could potentially mint multiple tokens without being restricted by the 9600 token limit.
5. This is a critical vulnerability as it can disrupt the intended token supply limit logic, leading to an oversupply of tokens, and consequently devalue the tokens or disrupt the contract's economic model.

The vulnerable code is:

```
function mint() public payable {
    uint256 supply = totalSupply();
    require(supply + 1 <= 9600, "Token limit reached");
    require(tokensSold + 1 <= weeklyLimit, "Weekly limit reached");

    if (msg.sender != owner()) {
        require(msg.value >= 0.1 ether, "Insufficient payment");
    }

    _safeMint(msg.sender, supply + 1);
    tokensSold += 1;
}
```

Medium vulnerabilities:

1 medium vulnerability was found in the smart contract.

Weekly Limit Not enforced properly:

The smart contract includes a mechanism designed to limit the number of tokens that can be minted on a weekly basis (**weeklyLimit**). However, during the audit, it was found that this limit is not being correctly enforced. The **mint()** function includes a check to ensure that the **tokensSold** plus one does not exceed the **weeklyLimit**. However, this check is not reset on a weekly basis. The lack of a reset mechanism means that, once the **weeklyLimit** is reached, no more tokens can be minted, contrary to the intent of a weekly limit. This vulnerability could impede the contract's operations once the **weeklyLimit** has been reached.

The vulnerable code is:

```
function mint() public payable {
    uint256 supply = totalSupply();
    require(supply + 1 <= 9600, "Token limit reached");
    require(tokensSold + 1 <= weeklyLimit, "Weekly limit reached");

    if (msg.sender != owner()) {
        require(msg.value >= 0.1 ether, "Insufficient payment");
    }

    _safeMint(msg.sender, supply + 1);
    tokensSold += 1;
}
```

Low vulnerabilities:

1 low vulnerability was found in the smart contract.

Lack of Events:

During the audit, an issue was identified in the smart contract related to a lack of event emission in key contract functions. Although this is not a critical security issue, it affects the transparency and traceability of transactions, which is vital for a robust, efficient, and transparent contract.

Events in a contract are essential tools that allow the efficient monitoring of contract operations. They log transactional information that can be useful for debugging and tracking contract activity over time. In the context of the contract in question, several important functions, such as the `mint()` function, do not emit events.

This can present several minor challenges:

1. It hinders the ability of users or external systems to listen for specific contract actions and respond to them.
2. It decreases the contract's transparency, making it more challenging to monitor contract activity, trace actions, or debug potential issues.
3. It limits the ability to create real-time systems that react to contract events, an increasingly common pattern in dApp development.

Disclaimer:

The smart contract was tested on a best-effort basis. The smart contract was analyzed with the best security practices known at the time of writing this report. Due to the fact that the total number of test cases is unlimited and the fact that new vulnerabilities in technologies are discovered every day, the audit report makes no guarantees on the security of the contract. It is recommended to not just rely on this audit report alone. A bug bounty program for this smart contract may be created to identify any vulnerabilities within the future.

Conclusion:

The smart contract was tested extensively both automatically and manually. Vulnerabilities were discovered within it. It is recommended that these vulnerabilities be resolved and additional security measures be undertaken to ensure future security such that of a creation of a bug bounty program.

It is also recommended to use a proxy mechanism in the smart contract to ensure that the smart contract code can be modified in the future.